

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 1 | SLO 1: Elements of Programming Languages (Code: 1011)

Q: 1. What is the syntax and semantics of the following Java statement: `int x = 5 + 3;`?

Ans: Syntax: The statement consists of a type specifier (`int`), an identifier variable name (`x`), the assignment operator (`=`), an arithmetic expression (`5 + 3`), and a statement terminator semicolon (`;`). It complies strictly with Java's Context-Free Grammar (CFG) rules for local variable declaration and initialization.

Semantics: At runtime, memory is allocated on the stack for a 32-bit signed two's complement integer. The binary addition operator computes the arithmetic sum of operands 5 and 3, yielding 8. The value 8 is then stored in the memory location referenced by variable `x`.

Q: 2. Identify lexical tokens in a simple Java program.

Ans: Consider the statement: `int count = 10;`

- **Keyword:** `int` (reserved word defining data type)
- **Identifier:** `count` (user-defined variable name)
- **Operator:** `=` (assignment operator)
- **Literal:** `10` (integer constant literal)
- **Separator/Punctuator:** `;` (statement terminator delimiter)

Q: 3. Modify a sample program to demonstrate the use of grammar rules in Java.

Ans: Java Demonstration:

Grammar rules dictate that statements must be inside methods, variables must be typed, and block scopes must be balanced.

```
public class GrammarDemo {
    public static void main(String[] args) {
        // Rule 1: Type declaration preceding identifier
        int base = 10;
        int height = 5;
        // Rule 2: Valid arithmetic expression and semicolon termination
        double area = 0.5 * base * height;
        // Rule 3: Balanced curly braces for block scope
        if (area > 0) {
            System.out.println("Calculated Area: " + area);
        }
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 1 | SLO 2: Syntax and Semantics (Code: 1012)

Q: 1. Which of the following is not an element of a programming language?

a) Syntax b) Algorithm c) Semantics d) Data types

Ans: Answer: b) Algorithm

Explanation: An algorithm is a language-independent step-by-step logic or procedure to solve a problem. In contrast, Syntax, Semantics, and Data Types are core formal elements inherent to programming language specifications.

Q: 2. Identify the correct pair of syntax and semantics:

a) Brackets → Loop b) Colon → Meaning c) if-else → Logic rule d) Semicolon → Statement terminator

Ans: Answer: d) Semicolon → Statement terminator

Explanation: The semicolon (;) is the concrete syntax in C/C++/Java, and its formal semantic meaning is marking the termination of an execution statement.

Q: 3. True or False: The semantics of a language define its grammatical rules.

Ans: Answer: False

Explanation: Grammatical rules define how valid expressions and statements are composed, which is the domain of **Syntax**. **Semantics** defines the meaning, runtime behavior, and evaluation of syntactically valid constructs.

Q: 4. What is the element responsible for variable declaration and type enforcement?

Ans: Answer: Data Type System / Type System

Explanation: The type system governs variable declarations, sets memory allocation requirements, defines permissible operations, and enforces compile-time and runtime type safety constraints.

Q: 5. Which element determines how functions and control flows are structured?

Ans: Answer: Control Structures and Formal Grammar (Control Flow Constructs)

Explanation: Control structures (such as conditional branching, loops, and subroutine call/return mechanisms) dictate the execution path and instruction sequencing of a program.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 1: Programming Language Theory (Code: 1021)

Q: 1. Syntax Check: Find and fix the syntax error in this Java code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10  
        System.out.println(num);  
    }  
}
```

Ans: Error: Missing statement terminator semicolon (;) after int num = 10 on line 3.

Corrected Code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10; // Fixed: added semicolon  
        System.out.println(num);  
    }  
}
```

Q: 2. Semantics Understanding: What will be the output of this code, and why?

```
int x = 5;  
int y = x;  
x = 10;  
System.out.println(y);
```

Ans: Output: 5

Explanation: In Java, primitive data types (like int) are passed and copied entirely by value. When int y = x; executes, the bitwise value of x (which is 5) is copied directly into y's distinct memory location. Modifying x = 10; afterwards has zero effect on y.

Q: 3. Pragmatics Insight: In Java, why is using ArrayList generally preferred over arrays for dynamic collections in modern programming environments?

Ans: Explanation:

- **Dynamic Resizing:** Standard Java arrays have fixed capacity defined at instantiation. ArrayList dynamically grows its internal storage buffer automatically as elements are appended.
- **Rich Utility Methods:** Provides built-in methods like add(), remove(), contains(), indexOf(), and clear().
- **Generics & Type Safety:** Enforces compile-time type checking with generic parameters (e.g., ArrayList<String>).
- **Interoperability:** Integrates seamlessly with the Java Collections Framework, iterators, and Java Stream API for functional filtering and mapping.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 2: Abstraction & Specification (Code: 1022)

Q: 1. Conceptual: Why is strong typing considered a significant design choice in modern programming languages?

Ans: Explanation: Strong typing ensures that the language strictly forbids implicit, unsafe type conversions (coercions) that could corrupt data or lead to unexpected behavior. It enhances code correctness, memory safety, predictability, and prevents subtle runtime bugs by forcing explicit conversions.

Q: 2. Analytical: Compare static typing and dynamic typing. How do these affect error detection and program reliability?

Ans: Comparison:

- **Static Typing (e.g., Java, C++):** Types are bound and verified at compile time. Syntax and type mismatch bugs are caught early before execution. IDEs provide accurate autocompletion and refactoring, yielding higher software reliability for enterprise systems.
- **Dynamic Typing (e.g., Python, JavaScript):** Types are associated with runtime values rather than variables. Offers rapid prototyping and concise syntax, but type errors surface only during runtime execution, requiring extensive test suite coverage.

Q: 3. Critical Thinking: Java and Python use different syntax styles. How does syntax design impact programmer productivity and code maintainability?

Ans: Analysis:

- **Python's Syntax:** Relies on meaningful indentation and omission of semicolons/parentheses. This minimizes visual boilerplate, speeding up initial development and prototyping.
- **Java's Syntax:** Utilizes explicit C-family braces ({}) and mandatory type declarations. While more verbose, it provides clear, unambiguous structural boundaries that facilitate maintainability and large-scale multi-developer collaboration.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 3 | SLO 1: Structured Programming & Bohm-Jacopini (Code: 1031)

Q: 1. Conceptual: Explain the three control structures identified in the Böhm-Jacopini theorem with simple Java examples.

Ans: The Böhm-Jacopini theorem proved that any computable function can be implemented using only three primary control structures:

1. **Sequence:** Ordered, step-by-step linear execution.
2. **Selection:** Conditional branching between paths (e.g., if-else or switch).
3. **Iteration:** Repetitive loop execution while a boolean predicate holds true (e.g., while, for).

```
// 1. Sequence
int a = 10;
int b = 20;
int sum = a + b;

// 2. Selection
if (sum > 25) {
    System.out.println("Sum is large");
} else {
    System.out.println("Sum is normal");
}

// 3. Iteration
for (int i = 0; i < 3; i++) {
    System.out.println("Iteration: " + i);
}
```

Q: 2. Analytical: Why did the Böhm-Jacopini theorem lead to the decline of the goto statement in modern programming languages?

Ans: Explanation: Prior to structured programming, code heavily relied on arbitrary jumps (goto), resulting in unstructured 'spaghetti code' that was impossible to trace, debug, or mathematically verify. Böhm-Jacopini provided the mathematical proof that goto is completely unnecessary, directly inspiring structured control statements with single-entry and single-exit points.

Q: 3. Application: Rewrite the following pseudocode (which uses goto) using structured constructs:

start:

x = x + 1

if (x < 10) goto start

Ans: Structured Equivalent in Java:

This construct executes the body at least once, making a do-while loop the most idiomatic equivalent:

```
int x = 0;
do {
    x = x + 1;
} while (x < 10);

// Alternatively using a standard while loop:
while (x < 10) {
    x = x + 1;
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 3 | SLO 2: Apply Control Structures (Code: 1032)

Q: 1. Sequence: Write a program to print your name and age.

Ans: Java Implementation:

```
public class SequenceDemo {  
    public static void main(String[] args) {  
        String name = "Sautrik Roy";  
        int age = 20;  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

Q: 2. Selection (if-else): Write code to check if a number is even or odd.

Ans: Java Implementation:

```
public class EvenOddDemo {  
    public static void main(String[] args) {  
        int number = 17;  
        if (number % 2 == 0) {  
            System.out.println(number + " is Even");  
        } else {  
            System.out.println(number + " is Odd");  
        }  
    }  
}
```

Q: 3. Iteration (loop): Print numbers from 1 to 5 using a loop.

Ans: Java Implementation:

```
public class LoopDemo {  
    public static void main(String[] args) {  
        System.out.println("Numbers from 1 to 5:");  
        for (int i = 1; i <= 5; i++) {  
            System.out.print(i + " ");  
        }  
        System.out.println();  
    }  
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 4 | SLO 1: Multiple Programming Paradigms (Code: 1041)

Q: 1. What is a programming paradigm?

Ans: Definition: A programming paradigm is a fundamental classification, overarching style, or architectural methodology of computer programming that defines how computation is structured and organized.

Q: 2. Name any three common programming paradigms.

Ans: 1. **Imperative / Procedural Paradigm** (e.g., C)
2. **Object-Oriented Paradigm (OOP)** (e.g., Java, C++)
3. **Functional Paradigm** (e.g., Haskell, Scala, Lisp)

Q: 3. Why is it useful for a language to support multiple paradigms?

Ans: Utility: Multi-paradigm languages (like modern Java, Python, and C++) allow developers to apply the optimal approach for each distinct task. For example, using declarative functional streams for high-speed data transformations while using object-oriented classes for complex business domain modeling.

Q: 4. Which paradigm focuses on real-world entities like objects and classes?

Ans: Answer: Object-Oriented Programming (OOP) Paradigm.

Q: 5. Which paradigm emphasizes step-by-step instructions to change program state?

Ans: Answer: Imperative Programming Paradigm.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 4 | SLO 2: Paradigm Hierarchy & Features (Code: 1042)

Q: 1. Conceptual: Arrange the following in a correct hierarchy:

• Imperative • Structured • Procedural • Object-Oriented

Ans: Correct Architectural Hierarchy:

1. **Imperative Programming** (Broadest category: state-manipulation paradigm)
2. ■■■ **Structured Programming** (Sub-discipline: eliminates goto using sequence, selection, iteration)
3. ■■■ **Procedural Programming** (Modular extension: groups instructions into routines/procedures)
4. ■■■ **Object-Oriented Programming** (Modern evolution: bundles state and behavior into encapsulated objects)

Q: 2. Definition Match: Match each paradigm to its defining feature:

• Functional • Object-Oriented • Procedural • Logic-Based

- a) Uses facts and rules for computation
- b) Focuses on objects and classes
- c) Uses functions to avoid mutable state
- d) Breaks programs into procedures or routines

Ans: Matches:

- **Functional** → c) Uses functions to avoid mutable state
- **Object-Oriented** → b) Focuses on objects and classes
- **Procedural** → d) Breaks programs into procedures or routines
- **Logic-Based** → a) Uses facts and rules for computation

Q: 3. True or False:

- a) All object-oriented programs are procedural.
- b) Structured programming is a subset of imperative programming.
- c) Functional programming encourages global variables.

Ans: Answers:

- a) **False:** OOP encapsulates state inside objects rather than relying on top-level procedural decomposition.
- b) **True:** Structured programming is a formalized subset of imperative programming that restricts control flow.
- c) **False:** Functional programming strictly avoids mutable state and global variables to prevent side effects.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 5 | SLO 1: Imperative Programming Paradigm (Code: 1051)

Q: 1. What is the main focus of the imperative programming paradigm?

Ans: Answer: The main focus is describing **HOW** a program operates through explicit, step-by-step instructions and statements that directly manipulate the program's memory state over time.

Q: 2. Name two key features of imperative languages.

Ans: 1. **Mutable State & Variables:** Program execution involves tracking and modifying values in named memory locations.
2. **Explicit Control Flow:** Direct ordering of execution using sequential statements, loops, conditional branching, and procedure calls.

Q: 3. Give one example of an imperative programming language.

Ans: Answer: C (or Pascal, Fortran, Java).

Q: 4. How is control flow managed in imperative programming?

Ans: Answer: Control flow is governed explicitly through conditional branching (if/else, switch), iteration constructs (for, while, do-while), and subroutine call/return mechanisms.

Q: 5. Is Java an imperative language? Explain in one sentence.

Ans: Answer: Yes, Java is fundamentally imperative because program logic is implemented through sequenced statements, variable state mutations, and explicit control flow structures within methods.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 5 | SLO 2: Programming Paradigm Hierarchy (Code: 1052)

Q: 1. Discuss the transition from Unstructured to Structured and Procedural Imperative Programming.

Ans: Discussion: Early imperative languages relied on unconditional jumps (goto), resulting in fragmented codebases with high maintenance overhead. The structured programming paradigm solved this by enforcing modular single-entry, single-exit constructs (sequence, selection, iteration). Procedural programming further refined this by abstracting repetitive code blocks into parameterized procedures and functions with local scoping.

Q: 2. Provide a Java code example illustrating the imperative paradigm with state mutation.

Ans: Java Implementation:

```
public class ImperativeStateDemo {  
    public static void main(String[] args) {  
        int total = 0; // Initial state  
        int[] numbers = {10, 20, 30, 40};  
        for (int i = 0; i < numbers.length; i++) {  
            total += numbers[i]; // State mutated step-by-step  
        }  
        System.out.println("Cumulative Total: " + total);  
    }  
}
```